02285 AI and MAS, F26
Exercises for week 5, 3/3-26

Exercise 1 (Iterated Width in a simple domain)

Consider the planning domain given by the following set of actions $\mathcal{A}$:

ACTION a_1
PRECONDITION p_1
EFFECT p_2

ACTION a_2
PRECONDITION p_1
EFFECT $p_2 \wedge p_3$

ACTION a_3
PRECONDITION $p_1 \wedge p_2$
EFFECT p_4

ACTION a_4
PRECONDITION $p_1 \wedge p_3$
EFFECT p_5

ACTION a_5
PRECONDITION $p_1 \wedge p_4$
EFFECT p_5

ACTION a_6
PRECONDITION $p_4 \wedge p_5$
EFFECT p_6

a) What is the set of ground atoms At in this planning domain? What is the size of T^1? And the size of T^6? Explain why $|T^6|$ (the size of T^6) is an upper bound on the size of the state space of any planning problem over $\mathcal{A}$.

b) Let $s_0 = \{p_1\}$, and define for each $i = 1, \ldots, 6$ the planning problem $P(p_i) = (\mathcal{A}, s_0, p_i)$. Show that all $P(p_i)$ have solutions.

c) Now run IW(1) on each of the planning problems $P(p_i)$ (it will suffice to draw one search tree for all of them, as they only differ in the goal), using the following conventions:

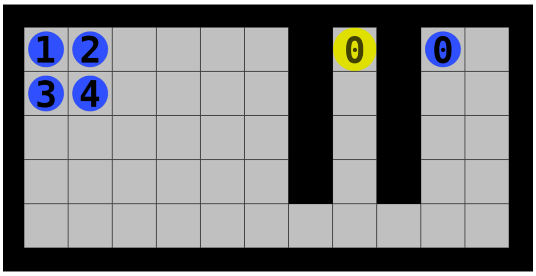
   (a) When expanding a state, we always add the child nodes to the frontier in order according to the index of the action that generated the child, i.e., we always consider the actions in order $a_1, a_2, a_3, \ldots$.

   (b) In the BFS expansion of the tree, we always expand nodes from left to right.

   Which of the planning problems are solved by IW(1), and why?

Exercise 2 (Iterated Width in MAPF)

Consider the following MAPF problem:



We can assume that states are described as sets of atoms of the form $AgentAt(agent, loc)$, where *agent* is the agent number (0–4) and *loc* is the location of the agent (e.g. represented as a 2D coordinate).

a) What are the optimal solutions to this problem? How long are the optimal solutions?

b) What will happen when running Iterated Width on this level? What is the lowest value of i for which $IW(i)$ will return a solution? (You should not actually run the algorithm by hand, but try to reason about what it will do.)

c) Is the solution returned optimal?

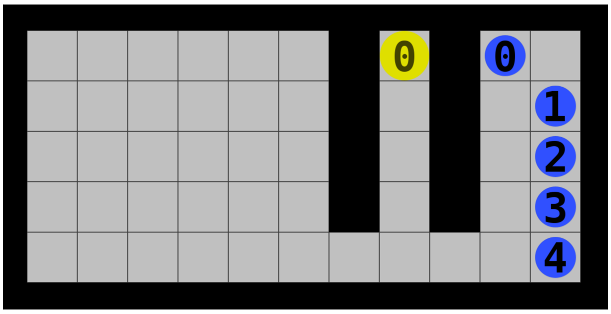
d) How large is the state space of the problem?

e) How large is T^1?

f) Approximately how many distinct states will be generated by IW before it finds a solution?

g) Approximately how many distinct states would greedy best-first graph search generate on this problem when using the Manhattan distance heuristic from agent to goal? Compare this to your number from the previous question.

h) Consider now instead the following variant of the problem:



Suppose first that when expanding a node, we always first consider the joint actions where as many of the individual actions as possible are noops. What is then the lowest value of i for which $IW(i)$ will return an optimal solution.

i) What is the lowest value of i for which $IW(i)$ is guaranteed to return an optimal solution to this level, no matter in which order we add children to the search tree?

j) Let i be the value from your previous question. Approximately how many distinct states does $IW(i)$ generate to find the optimal solution?

Exercise 3 (Repeated actions in IW)

a) Consider the search tree for $IW(1)$ that you produced in Exercise 1. Note that each action a_i appear at most once in the tree. Now argue that for *any* planning problem, the search tree of $IW(1)$ will contain each (ground) action at most once, i.e., there will be at most one edge labelled with that action.

b) An action b is called the *inverse* of an action a, if for any state s in which a is applicable, executing a followed by b in s leads back to s, i.e., if

$$\text{Result}(\text{Result}(s,a),b) = s.$$

When the inverse of a exists, we usually denote it by a^{-1}. Note that each action in the hospital domain has an inverse. For instance, the inverse of $Move(N)$ is $Move(S)$.

Now prove that for any planning problem with inverses, if the search tree of $IW(1)$ has an edge labelled by an action a, then no other edge is labelled by a or a^{-1}. You are here encouraged to try to give a detailed mathematical proof.

c) It follows from the previous question that $IW(1)$ can never produce a solution where an action is performed twice, or performed once and then later inversed. Use this insight to design a level in the hospital domain that cannot be solved by $IW(1)$. Try to make the level as simple as possible.

d) Will the level you designed in the previous question necessarily be solved by $SIW(1)$, i.e., SIW restricted to width 1? By "necessarily solved" we mean that it will be solved independently of the order in which subgoals are considered, independently of the order in which actions are considered during node expansion and independently of the order in which nodes at the same level of the tree are expanded. If the answer to the question is yes, then try to design another simple level that also $SIW(1)$ cannot necessarily solve.

e) Did the level you designed in the previous question have 1 or multiple subgoals? If multiple, can you design a level with a single subgoal that $IW(1)$ cannot solve?

What we learn from this exercise is that in general we should not expect too much from $IW(1)$ in the hospital domain, since actions cannot be repeated or inversed, meaning that we for instance cannot move back along a path that we earlier moved forward along (unless we then also carry a box one of the times). Such moving back and forth can be expected to happen quite regularly in single-agent levels with many boxes. Think for instance about how often a warehouse robot might use the same path. We should then at least move to $SIW(1)$ if we want something that works well in the hospital domain. However, as the last question shows, even $SIW(1)$ can be fooled. But now note that it never makes sense to do the exact same action twice or undo an earlier action unless something else has changed in the meantime (this is true for *any* domain, also in real life). That 'something' is not necessarily an explicit subgoal in the formulation of the problem, but it could easily be an implicit subgoal. For instance in the hospital domain, an implicit subgoal could be 'move box out of path', if a box is initially blocking a path that is required to achieve another (sub)goal. Adding such implicit subgoals as explicit subgoals can then actually help to make SIW perform better, in particular if the subgoals are also considered in a clever order. Note the relation to HTN: One could consider implicit subgoals such as 'move box out of path' as high-level actions, and then first finding a high-level plan using actions such as 'move to box', 'move box out of path' and 'move box to goal' can help significantly in reducing the search efforts.

Exercise 4 (Improving IW?)

Above we saw that the order of adding children can matter. Consider the following alternative to IW that tries to remedy this:

Definition. *$IW'(i)$ is the algorithm that runs like breadth-first search except a child state s is only added to the frontier if it contains at least one new T^i state that is not already contained in any tree node at strictly lower depths of the tree.*

a) Will $IW'(1)$ find an optimal solution to any MAPF problem where only 1 agent has a goal, and the optimal path is initially not obstructed?

b) $IW(i)$ has the property that the number of states it generates is at most exponential in i, more precisely, the number of distinct states it generates is at most $|T^i| \leq (|At| + 1)^i$. Is the same true for $IW'(i)$?

Exercise 5 (OPTIONAL) (Proof of correctness and completeness of IW)

The slides stated the following theorem.

Theorem. Let P be a planning problem of width w. Then $IW(w)$ solves it optimally.

Prove the theorem. *Hint*: Part of the proof will be by induction on the depth of the search tree.